

HttpSession & Session Management in Servlet

1. Why Do We Need Sessions?

When a user logs in, we may need to remember information such as:

- Username
- Email
- City
- User ID

Normally, this information can be sent from one page to another using the **Request object**.

However, a Request object is available only for **one request-response cycle**.

For example:

```
Login Page
  ↓
Request
  ↓
Profile Page
```

After the request is completed, that particular request data is no longer available for the next request.

So, if the user moves from:

```
Login → Profile → Home → About
```

the application needs another mechanism to remember the user's information.

Solution

We use **HttpSession**.

2. What is HttpSession?

Definition

HttpSession: `HttpSession` is used in Servlets to store and maintain **user-specific information across multiple requests**.

In simple words:

Session helps the server remember a user while they are using the application.

For example, after login:

```
Username = Deepak  
Email    = deepak@gmail.com  
City     = Pune
```

These values can be stored in the session and accessed from different pages.

Remember

Request → One request

Session → Multiple requests of the same user

3. How to Implement HttpSession

There are three basic steps.

Step 1: Create or Get a Session

Use the `getSession()` method of `HttpServletRequest`.

```
HttpSession hs = request.getSession();
```

What does `getSession()` do?

It:

- Returns the existing session if the user already has one.
- Creates a new session if one does not exist.

So:

```
request.getSession();
```

means:

Get the existing session or create a new one.

Step 2: Store Data in the Session

Use `setAttribute()` to store information.

```
hs.setAttribute("name_key", "Deepak Pawar");
```

Here:

- "name_key" → Key
- "Deepak Pawar" → Value

The data is stored in **key-value form**.

Example:

Key	Value
name_key	Deepak Pawar
email_key	deepak@gmail.com
city_key	Pune

General Syntax

```
session.setAttribute("key", value);
```

Step 3: Retrieve Data from the Session

Use `getAttribute()` to retrieve stored information.

```
String name = (String) hs.getAttribute("name_key");
```

Here:

- "name_key" is used to find the stored value.
- The returned value is cast to `String`.

General Syntax

```
Object value = session.getAttribute("key");
```

Example:

```
String name = (String) hs.getAttribute("name_key");
```

4. How to Destroy a Session

When the user clicks **Logout**, we usually need to remove their session information.

There are two important methods.

A. Remove a Specific Attribute

Use:

```
hs.removeAttribute("name_key");
```

This removes only the particular attribute.

For example:

Before :

```
name_key  → Deepak  
email_key → deepak@gmail.com  
city_key  → Pune
```

After:

```
hs.removeAttribute("name_key");
```

The result is:

```
email_key → deepak@gmail.com  
city_key  → Pune
```

Only `name_key` is removed.

B. Destroy the Entire Session

Use:

```
hs.invalidate();
```

`invalidate()` destroys the complete session.

It is commonly used when the user logs out.

Example:

```
HttpSession hs = request.getSession();  
hs.invalidate();
```

Important Difference

Method	Purpose
<code>removeAttribute()</code>	Removes one particular attribute
<code>invalidate()</code>	Destroys the complete session

Important for Exam:

`removeAttribute()` removes specific data, while `invalidate()` destroys the entire session.

5. Session Timeout

A session can also be destroyed automatically after a certain period of inactivity.

For example:

```
User active  
  ↓  
No activity  
  ↓  
Timeout period  
  ↓  
Session expires
```

The timeout period can be configured for the application.

Remember:

A session can expire automatically when the user remains inactive for a specified period.

6. Internal Working of Session Tracking

Now consider that many users are using the same application.

For example:

Client-1 → Deepak
Client-2 → Rahul

The server needs to know:

Which data belongs to which user?

This is handled using a **Session ID (SID)**.

Definition

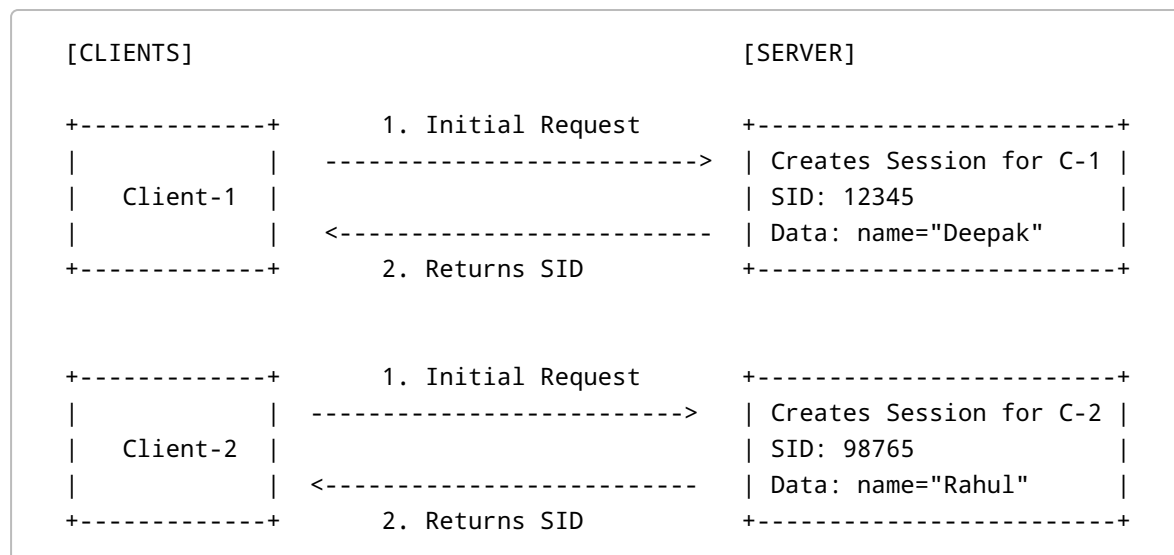
Session ID: A unique identifier used to identify a particular user's session.

For example:

Client-1 → SID: 12345
Client-2 → SID: 98765

Because the Session IDs are different, the server can keep their session data separate.

7. Session Tracking Diagram



8. How Session Tracking Works

Step 1: Client Sends the First Request

Suppose **Client-1** visits the application.

```
Client-1
  |
  | Initial Request
  ↓
Server
```

The server checks whether the client already has a session.

If there is no existing session, the server creates one.

```
Session
SID = 12345
```

Step 2: Server Stores User Data

The server can store the user's information in that session.

```
SID: 12345
Name: Deepak
```

So the server now has a session associated with Client-1.

Step 3: Server Returns the Session ID

The server sends the session information back to the client.

```
Server
  |
  | SID = 12345
  ↓
Client-1
```

The client can use this Session ID for later requests.

Step 4: Client Sends Another Request

Suppose Client-1 now opens the **Home** page.

The client sends another request containing its session information.

```
Client-1
SID = 12345
  |
  | Request
  ↓
Server
```

The server sees:

```
SID = 12345
```

It can therefore find the session belonging to Client-1.

```
SID: 12345
Name: Deepak
```

The server knows that the request belongs to **Deepak's session**.

9. Multiple Clients

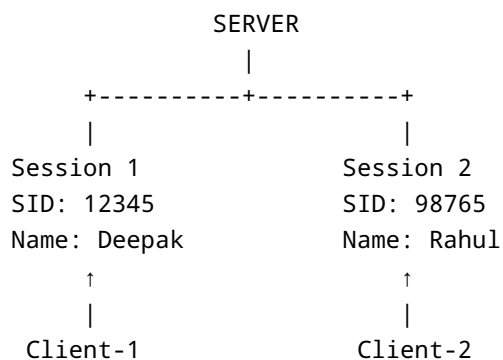
Suppose Client-2 also uses the application.

The server creates a separate session for Client-2.

```
Client-1 → SID: 12345 → Name: Deepak
```

```
Client-2 → SID: 98765 → Name: Rahul
```

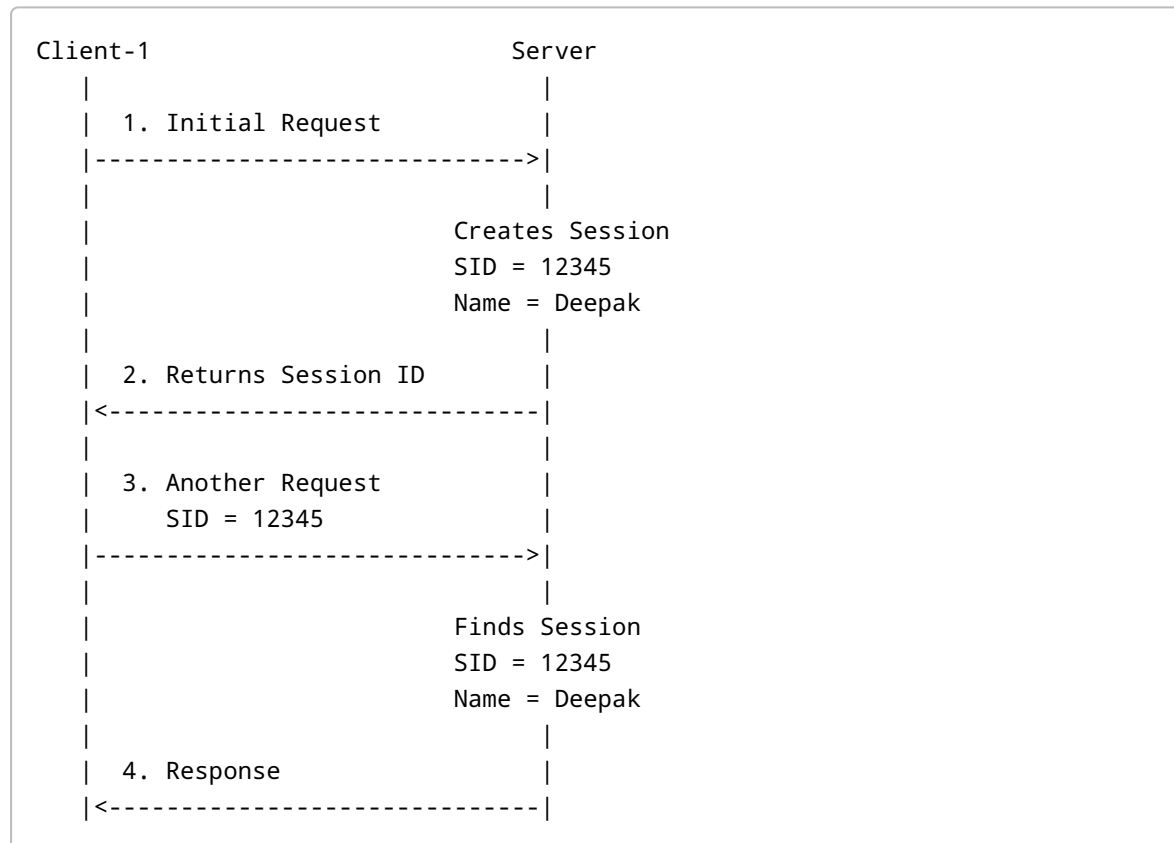
The server can distinguish them because their Session IDs are different.



Therefore:

- Client-1 gets Deepak's data.
- Client-2 gets Rahul's data.
- Their session data does not get mixed.

10. Complete Session Flow



11. Important HttpSession Methods

Method	Meaning
<code>request.getSession()</code>	Creates or gets a session
<code>setAttribute()</code>	Stores data
<code>getAttribute()</code>	Retrieves data
<code>removeAttribute()</code>	Removes one attribute
<code>invalidate()</code>	Destroys the complete session

12. Example: Login and Logout

Login

```
HttpSession hs = request.getSession();  
  
hs.setAttribute("name_key", "Deepak Pawar");
```

The username is now stored in the session.

Access Username on Another Page

```
HttpSession hs = request.getSession();  
  
String name = (String) hs.getAttribute("name_key");
```

The application can retrieve the username from the session.

Logout

```
HttpSession hs = request.getSession();  
  
hs.invalidate();
```

The complete session is destroyed.

13. Request vs HttpSession

Feature	Request	HttpSession
Scope	One request	Multiple requests
Purpose	Transfer request data	Maintain user information
Lifetime	Short/temporary	Until invalidated or expired
Used for	Form/request data	Login and user-specific data
Identification	Request	Session ID

Commonly Confused

Request:

Used for data needed during one request.

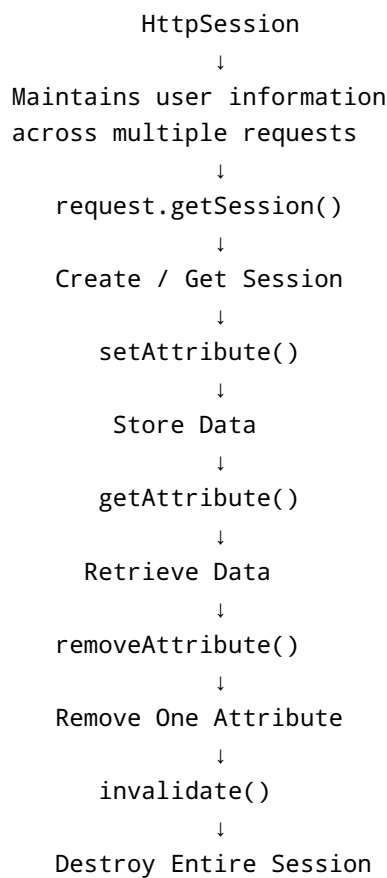
Session:

Used for data that needs to remain available across multiple requests of the same user.

14. Important for Exam

- **HttpSession** is used for **session management** in Servlets.
 - It allows user information to be maintained across multiple requests.
 - `request.getSession()` creates or retrieves a session.
 - `setAttribute()` is used to store data.
 - `getAttribute()` is used to retrieve data.
 - `removeAttribute()` removes a particular attribute.
 - `invalidate()` destroys the complete session.
 - **Session ID (SID)** identifies a user's session.
 - Different users have different sessions and Session IDs.
 - A session can expire after a period of inactivity.
 - Sessions are commonly used for **login/logout functionality**.
-

15. Quick Revision



Key Points to Remember

- **Request** = temporary
- **Session** = maintains user data
- **SID** = identifies the session
- `getSession()` = create/get session
- `setAttribute()` = store data
- `getAttribute()` = retrieve data
- `removeAttribute()` = remove one value
- `invalidate()` = destroy entire session
- **Timeout** = automatic session expiration after inactivity

Summary

`HttpSession` is used to maintain a user's information across multiple requests in a Servlet application.

The basic flow is:

```
getSession()
  ↓
Create/Get Session
  ↓
setAttribute()
  ↓
Store User Data
  ↓
getAttribute()
  ↓
Retrieve User Data
  ↓
invalidate()
  ↓
Destroy Session during Logout
```

The server uses a **Session ID** to identify the correct session for each client. This allows multiple users to use the same application without mixing their personal session data.

Video Info-----

Video title: #10 HttpSession Session Management in Servlet || Session Tracking Hindi || Servlet and JSP

Video link: https://youtu.be/6ASoqqSZY_g?si=HwvQG257D_4k238g